

Touchdown Labs. Twenty months at the kernel layer.

William Chen — Founder & CEO, Touchdown Labs. Twenty months studying **CUDA kernel generation and AMD GPU kernels** — through chip-design study, AMD developer hackathons, and **joint IC-design research with Taiwan's NYCU** — building the inference-optimization company that measures **cost per successful AI task** from the kernel up to the token bill.

Team-to-problem alignment: our day job runs from CUDA kernels to token bills. This lab teaches students the same discipline — on the coding agent they already use — with the same evidence standard we hold our own kernel work to.



AI EXPO TAIWAN 2026 · GLOBAL PAVILION
BOOTH · EXHIBITOR

PROOF OF WORK · WON THE TRACK Won Mercor's OpenEnv Hackathon track (San Francisco, March 2026) with an RL environment for automated CUDA kernel generation — the first public event built around RL environments and post-training. Writeup: Automated CUDA Is Almost Here.	HARDWARE FIELD WORK Exhibited at AI EXPO Taiwan (Global Pavilion booth, Taipei, June 2026) and flew to Computex 2026 — writeup: Seeing the Physical Layer of the AI Factory — attended GTC 2026. Joint IC-design research with NYCU and a growing research-partner network, plus MI355X kernel experiments and AMD developer hackathons.	THIS BUILD, ALREADY VERIFIED 10 / 10 local tests passing, frozen acceptance contract, and a live Butterbase data write (receipt row <code>e5ff8fc6...9454</code>). Fixture and live states are always labeled.
--	---	--

Run one coding task. See every token the agent remembered, recomputed, and wasted — then fix it.

THE PAIN

Coding agents hide the bill inside the loop. One small retry-bug fix loaded **40,176 characters** of harness context — **17,420 of them repeated** (43%) across 14 tool calls. Students see a patch; nobody teaches them what the system paid, or how to make it cheaper without breaking the tests.

THE BUILT SOLUTION

A browser lab on a frozen, reproducible coding task. A three-minute, nine-stage lesson: classify context lifetime, apply one optimization policy at a time, watch the engine decision flip from local MLX-LM to vLLM on Nebius at 100 users, follow text → tokens → prefill → KV cache → HBM, and export a signed optimization receipt. EverOS memory makes the next lesson adapt; Butterbase persists the evidence.

CONTEXT AFTER ONE POLICY

–42%

40,176 → 23,210 characters on the same task, same acceptance test still passing.

DETERMINISTIC FIXTURE · LABELED

REPEATED CONTEXT

–70%

17,420 → 5,180 repeated characters once stable prefix + tool-output projection are applied.

DETERMINISTIC FIXTURE · LABELED

MEMORY VS FULL REPLAY

–92%

~4,210 → ~323 estimated tokens by retrieving an EverOS Case instead of replaying the full trajectory.

LOCAL ESTIMATE · NOT A PROVIDER MEASUREMENT

LEARN HOW TO DO THIS — THE LAB SHIPS ITS OWN CURRICULUM

- **Accounting first:** `total input = input_tokens + cache_creation_input_tokens + cache_read_input_tokens` (Claude Code) — the repo's [agent-optimizer skill](#) makes every claim a before/after receipt.
- **Take it home:** run the same KV-cache loop on your own laptop — [Set Up a Local OpenClaw or Hermes KV-Cache Learning Loop with Mac MLX](#) (mirrored in this repo).